

Operating System

Unit 1

Content

Introduction to Operating System

- Operating systems concepts
- Types of operating systems
- Evolution of operating systems
- Different views of the operating system
- Operating system services
- System calls
- Types of system calls
- Operating system structure
- Layered approach
- Micro kernels
- Virtual machines.

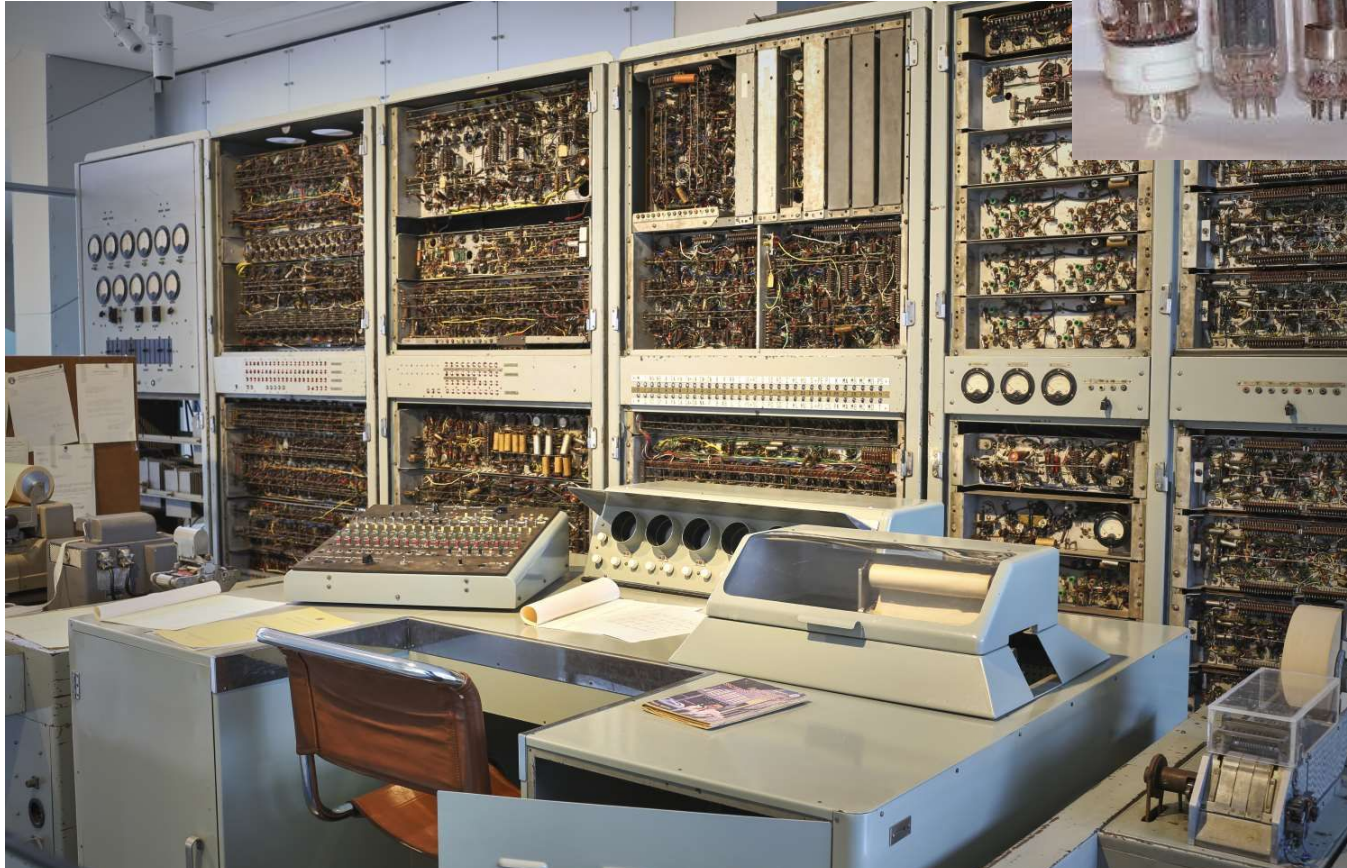
Operating systems concepts

- An Operating System(OS) is software that manages and handles the hardware and software resources of a computer system.
- It provides interaction between users of computers and computer hardware. An operating system is responsible for managing and controlling all the activities and sharing of computer resources.
- An operating system is a low-level Software that includes all the basic functions like processor management, memory management, Error detection, etc.

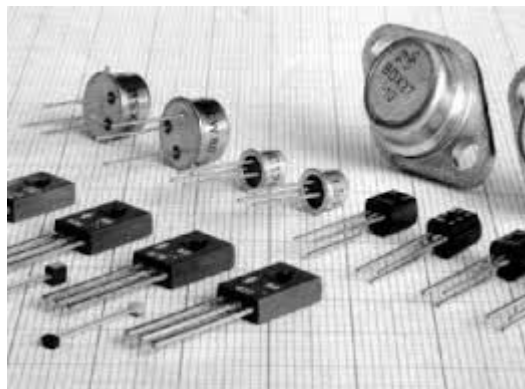
History of Operating System

Generation	Year	Electronic device used	Types of OS Devices
First	1945-55	Vacuum Tubes	Plug Boards
Second	1955-65	Transistors	Batch Systems
Third	1965-80	Integrated Circuits(IC)	Multiprogramming
Fourth	Since 1980	Large Scale Integration	PC

First Generation



Second Generation



Third Generation Computer

Time: 1964 - 1971



Fourth Generation Computer

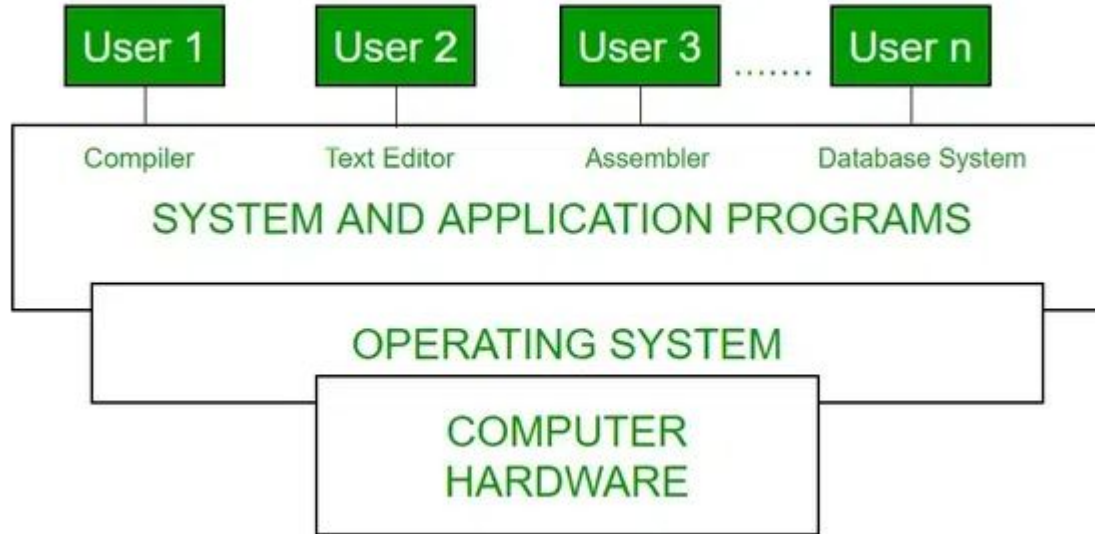


Operating System - Functions

To brief, Following are some of important functions of an operating System

- Process Management
- I/O Device Management
- File Management
- Network Management
- Main Memory Management
- Secondary Storage Management
- Security Management
- Command Interpreter System
- Control over system performance
- Job Accounting
- Error Detection and Correction
- Coordination between other software and users
- Many more other important tasks

The process operating system as User Interface



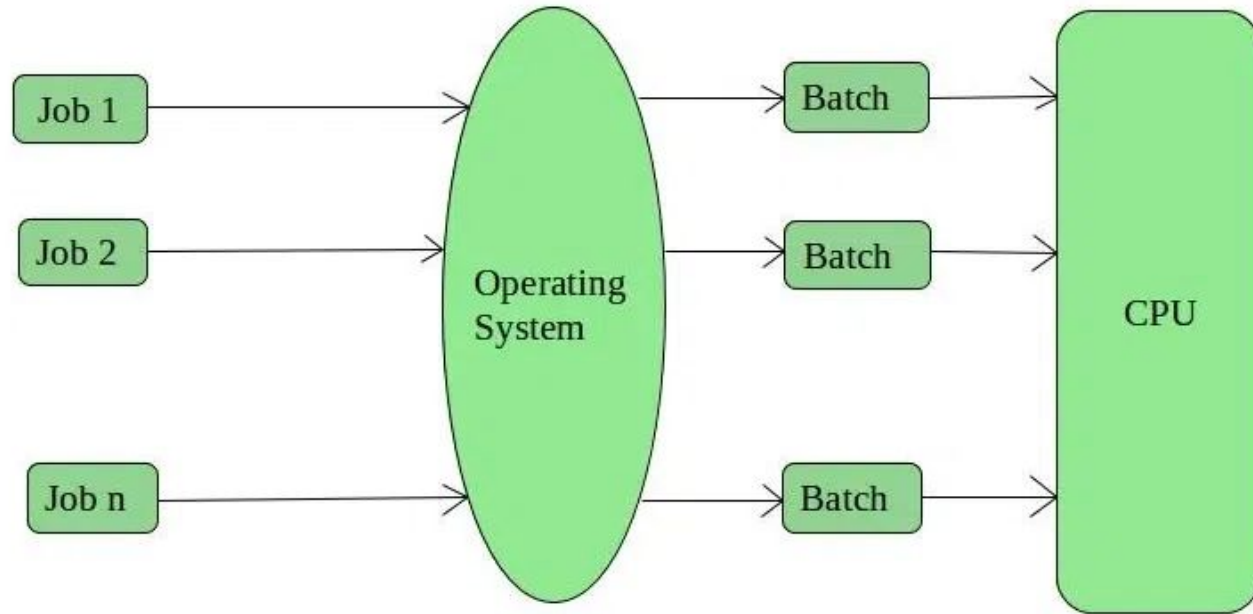
Types of Operating Systems

There are several types of Operating Systems which are mentioned below.

- Batch Operating System
- Multi-Programming System
- Multi-Processing System
- Multi-Tasking Operating System
- Time-Sharing Operating System
- Distributed Operating System
- Network Operating System
- Real-Time Operating System

Batch Operating System

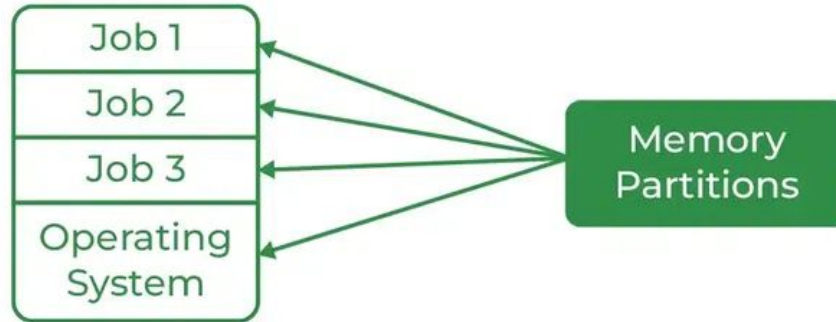
Examples of Batch Operating Systems: Payroll Systems, Bank Statements, etc.



Multi-Programming Operating System

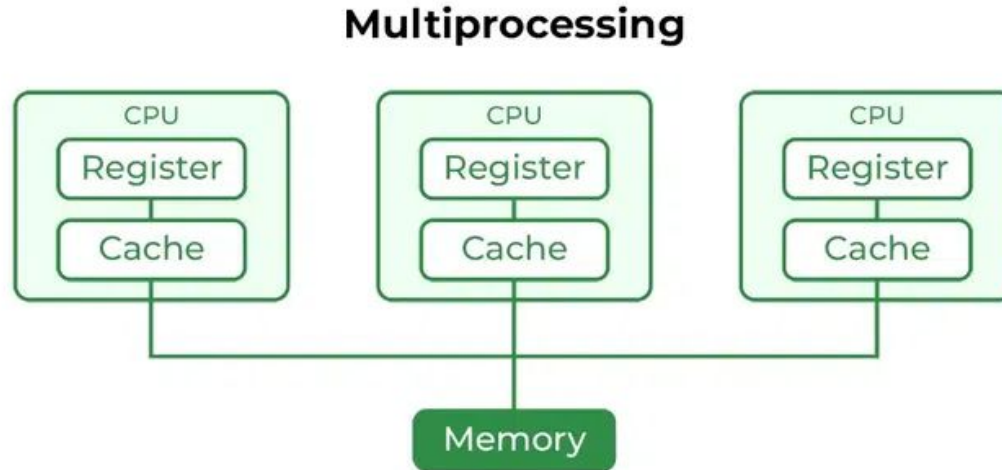
Multiprogramming Operating Systems can be simply illustrated as more than one program is present in the main memory and any one of them can be kept in execution. This is basically used for better execution of resources.

Multiprogramming



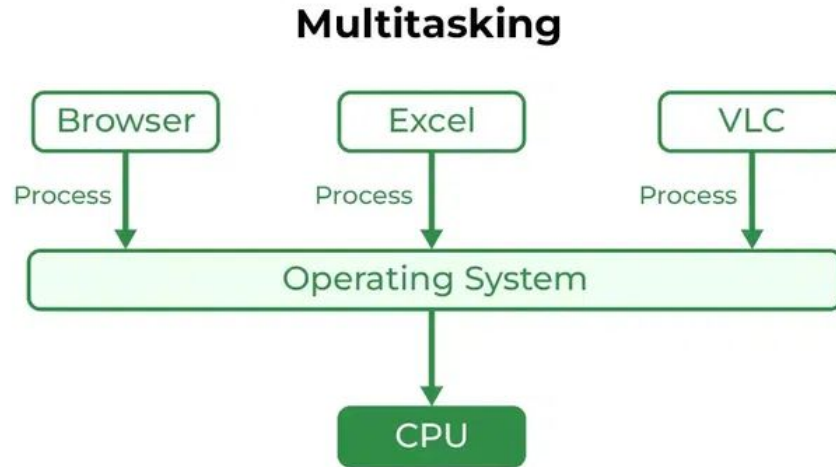
Multi-Processing Operating System

Multi-Processing Operating System is a type of Operating System in which more than one CPU is used for the execution of resources. It better the throughput of the System.



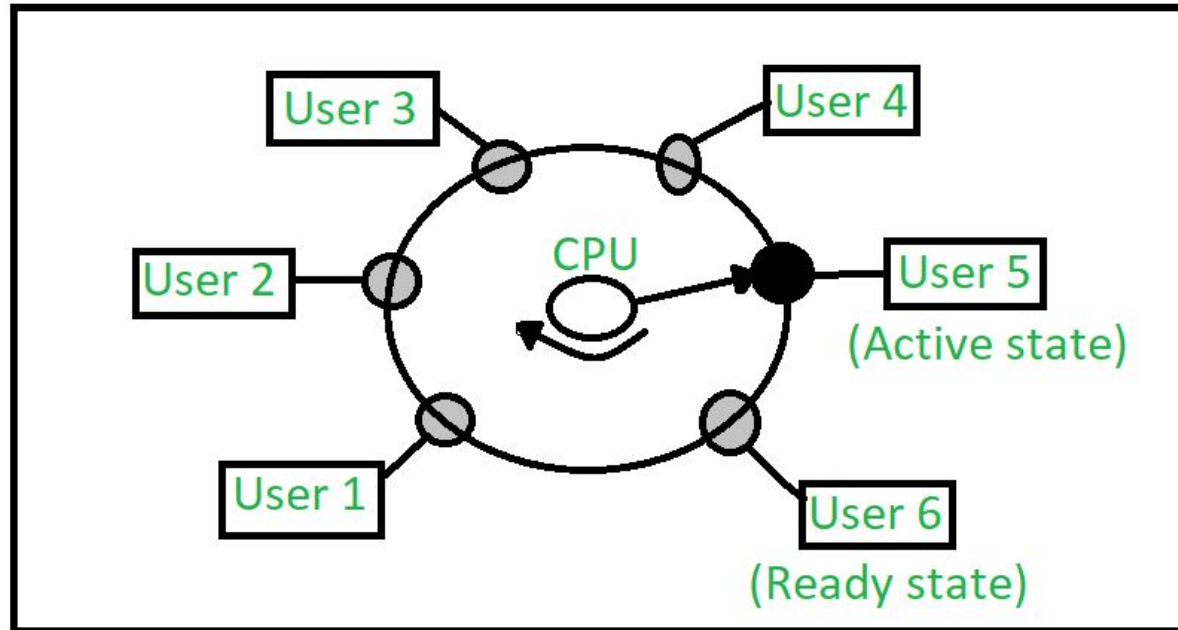
Multi-Tasking Operating System

Multitasking Operating System is simply a multiprogramming Operating System with having facility of a Round-Robin Scheduling Algorithm. It can run multiple programs simultaneously.



Time-Sharing Operating Systems

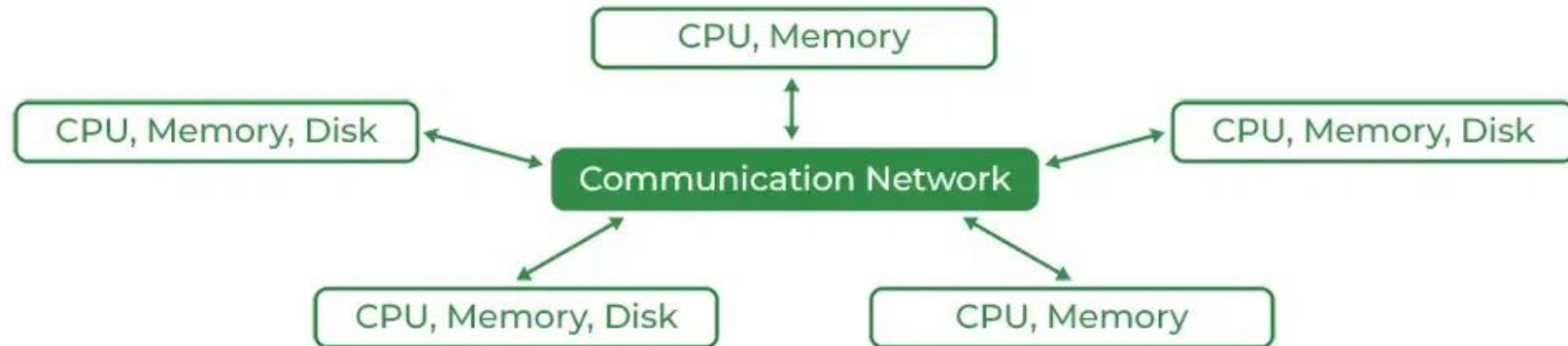
Each task is given some time to execute so that all the tasks work smoothly. Each user gets the time of the CPU as they use a single system. These systems are also known as Multitasking Systems.



Distributed Operating System

Various autonomous interconnected computers communicate with each other using a shared communication network. Independent systems possess their own memory unit and CPU. These are referred to as loosely coupled systems or distributed systems.

Architecture of Distributed OS



Network Operating System

These systems run on a server and provide the capability to manage data, users, groups, security, applications, and other networking functions. These types of operating systems allow shared access to files, printers, security, applications, and other networking functions over a small private network. One more important aspect of Network Operating Systems is that all the users are well aware of the underlying configuration, of all other users within the network, their individual connections, etc.

Examples of Network Operating Systems are Microsoft Windows Server 2023, Microsoft Windows Server 2008, UNIX, Linux, Mac OS X, Novell NetWare, BSD, etc.

Real-Time Operating System

The time interval required to process and respond to inputs is very small. This time interval is called response time. Real-time systems are used when there are time requirements that are very strict like missile systems, air traffic control systems, robots, etc.

Types of Real-Time Operating Systems

- **Hard Real-Time Systems:** Hard Real-Time OSs are meant for applications where time constraints are very strict and even the shortest possible delay is not acceptable. These systems are built for saving life like automatic parachutes or airbags which are required to be readily available in case of an accident. Virtual memory is rarely found in these systems.
- **Soft Real-Time Systems:** These OSs are for applications where time-constraint is less strict.
- Examples of Real-Time Operating Systems are Scientific experiments, medical imaging systems, industrial control systems, weapon systems, robots, air traffic control systems, etc.

Evolution of operating systems

Operating Systems have evolved in past years. It went through several changes before getting its original form. These changes in the operating system are known as the evolution of operating systems. OS improve itself with the invention of new technology. Basically , OS added the feature of new technology and making itself more powerful. Let us see the evolution of operating system year-wise in detail:

1. No OS – (0s to 1940s)
2. Batch Processing Systems -(1940s to 1950s)
3. Multiprogramming Systems -(1950s to 1960s)
4. Time-Sharing Systems -(1960s to 1970s)
5. Introduction of GUI -(1970s to 1980s)
6. Networked Systems – (1980s to 1990s)
7. Mobile Operating Systems – (Late 1990s to Early 2000s)
8. AI Integration – (2010s to ongoing)

No OS – (0s to 1940s)

As we know that before 1940s, there was no use of OS . Earlier, people are lacking OS in their computer system so they had to manually type instructions for each tasks in machine language(0-1 based language) . And at that time , it was very hard for users to implement even a simple task. And it was very time consuming and also not user-friendly . Because not everyone had that much level of understanding to understand the machine language and it required a deep understanding.

Batch Processing Systems -(1940s to 1950s)

With the growth of time, batch processing system came into the market .Now Users had facility to write their programs on punch cards and load it to the computer operator. And then operator make different batches of similar types of jobs and then serve the different batch(group of jobs) one by one to the CPU .CPU first executes jobs of one batch and then jump to the jobs of other batch in a sequence manner.

Multiprogramming Systems -(1950s to 1960s)

Multiprogramming was the first operating system where actual revolution began. It provide user facility to load the multiple program into the memory and provide a specific portion of memory to each program. When one program is waiting for any I/O operations (which take much time) at that time the OS give permission to CPU to switch from previous program to other program(which is first in ready queue) for continuous execution of program with interrupt.

Time-Sharing Systems -(1960s to 1970s)

Time-sharing systems is extended version of multiprogramming system. Here one extra feature was added to avoid the use of CPU for long time by any single program and give access of CPU to every program after a certain interval of time. Basically OS switches from one program to another program after a certain interval of time so that every program can get access of CPU and complete their work.

Introduction of GUI -(1970s to 1980s)

With the growth of time, Graphical User Interfaces (GUIs) came. First time OS became more user-friendly and changed the way of people to interact with computer. GUI provides computer system visual elements which made user's interaction with computer more comfortable and user-friendly. User can just click on visual elements rather than typing commands. Here are some feature of GUI in Microsoft's windows icons, menus and windows.

Networked Systems – (1980s to 1990s)

At 1980s, the craze of computer networks is at its peak. A special type of Operating Systems needed to manage the network communication. The OS like Novell NetWare and Windows NT were developed to manage network communication which provide users facility to work in collaborative environment and made file sharing and remote access very easy.

Mobile Operating Systems – (Late 1990s to Early 2000s)

Invention of smartphones create a big revolution in software industry, To handle the operation of smartphones , a special type of operating systems were developed. Some of them are : iOS and Android etc. These operating systems were optimized with the time and became more powerful.

AI Integration – (2010s to ongoing)

With the growth of time, Artificial intelligence came into picture. Operating system integrates features of AI technology like Siri, Google Assistant, and Alexa and became more powerful and efficient in many way. These AI features with operating system create a entire new feature like voice commands, predictive text, and personalized recommendations.

Note: The above mentioned OS basically tells how the OS evolved with the time by adding new features but it doesn't mean that only new generation OS are in use and previously OS system are not in use, according to the need, all these OS are still used in software industry.

Different views of the operating system

- The operating system can be observed from the point of view of the user or the system. This is known as the user view and system view respectively.
- An operating system is a framework that enables user application programs to interact with system hardware.
- The operating system does not perform any functions on its own, but it provides an atmosphere in which various programs and apps can do useful work.
- Mainly two types of views of the operating system.
 - User view
 - System view

System View

- The OS may also be viewed as just a resource allocator.
- A computer system comprises various sources, such as hardware and software, which must be managed effectively.
- The operating system manages the resources, decides between competing demands, controls the program execution, etc.
- According to this point of view, the operating system's purpose is to maximize performance.
- The operating system is responsible for managing hardware resources and allocating them to programs and users to ensure maximum performance.

Operating system services

An Operating System provides services to both the users and to the programs.

- It provides programs an environment to execute.
- It provides users the services to execute the programs in a convenient manner.

Following are a few common services provided by an operating system –

- Program execution
- I/O operations
- File System manipulation
- Communication
- Error Detection
- Resource Allocation
- Protection

Program Execution

- Operating systems handle many kinds of activities from user programs to system programs like printer spooler, name servers, file server, etc. Each of these activities is encapsulated as a process.
- A process includes the complete execution context (code to execute, data to manipulate, registers, OS resources in use).
- Following are the major activities of an operating system with respect to program management Loads a program into memory.
 - Executes the program.
 - Handles program's execution.
 - Provides a mechanism for process synchronization.
 - Provides a mechanism for process communication.
 - Provides a mechanism for deadlock handling.

Input Output Operations

- An I/O subsystem comprises of I/O devices and their corresponding driver software. Drivers hide the peculiarities of specific hardware devices from the users.
- An Operating System manages the communication between user and device drivers.
- I/O operation means read or write operation with any file or any specific I/O device.
- Operating system provides the access to the required I/O device when required.

File System manipulation

- A file represents a collection of related information.
- Computers can store files on the disk (secondary storage), for long-term storage purpose.
- Examples of storage media include magnetic tape, magnetic disk and optical disk drives like CD, DVD. Each of these media has its own properties like speed, capacity, data transfer rate and data access methods.
- A file system is normally organized into directories for easy navigation and usage. These directories may contain files and other directions.

File System manipulation . . .

Following are the major activities of an operating system with respect to file management

- Program needs to read a file or write a file.
- The operating system gives the permission to the program for operation on file.
- Permission varies from read-only, read-write, denied and so on.
- Operating System provides an interface to the user to create/delete files.
- Operating System provides an interface to the user to create/delete directories.
- Operating System provides an interface to create the backup of file system.

Communication

- In case of distributed systems which are a collection of processors that do not share memory, peripheral devices, or a clock, the operating system manages communications between all the processes.
- Multiple processes communicate with one another through communication lines in the network.
- The OS handles routing and connection strategies, and the problems of contention and security.

Communication ...

Following are the major activities of an operating system with respect to communication –

- Two processes often require data to be transferred between them
- Both the processes can be on one computer or on different computers, but are connected through a computer network.
- Communication may be implemented by two methods, either by Shared Memory or by Message Passing.

Error Detection

Errors can occur anytime and anywhere. An error may occur in CPU, in I/O devices or in the memory hardware.

Following are the major activities of an operating system with respect to error handling –

- The OS constantly checks for possible errors.
- The OS takes an appropriate action to ensure correct and consistent computing.

Resource Allocation

In case of multi-user or multi-tasking environment, resources such as main memory, CPU cycles and files storage are to be allocated to each user or job.

Following are the major activities of an operating system with respect to resource management –

- The OS manages all kinds of resources using schedulers.
- CPU scheduling algorithms are used for better utilization of CPU.

Protection

Considering a computer system having multiple users and concurrent execution of multiple processes, the various processes must be protected from each other's activities.

Protection refers to a mechanism or a way to control the access of programs, processes, or users to the resources defined by a computer system.

Following are the major activities of an operating system with respect to protection –

- The OS ensures that all access to system resources is controlled.
- The OS ensures that external I/O devices are protected from invalid access attempts.
- The OS provides authentication features for each user by means of passwords.

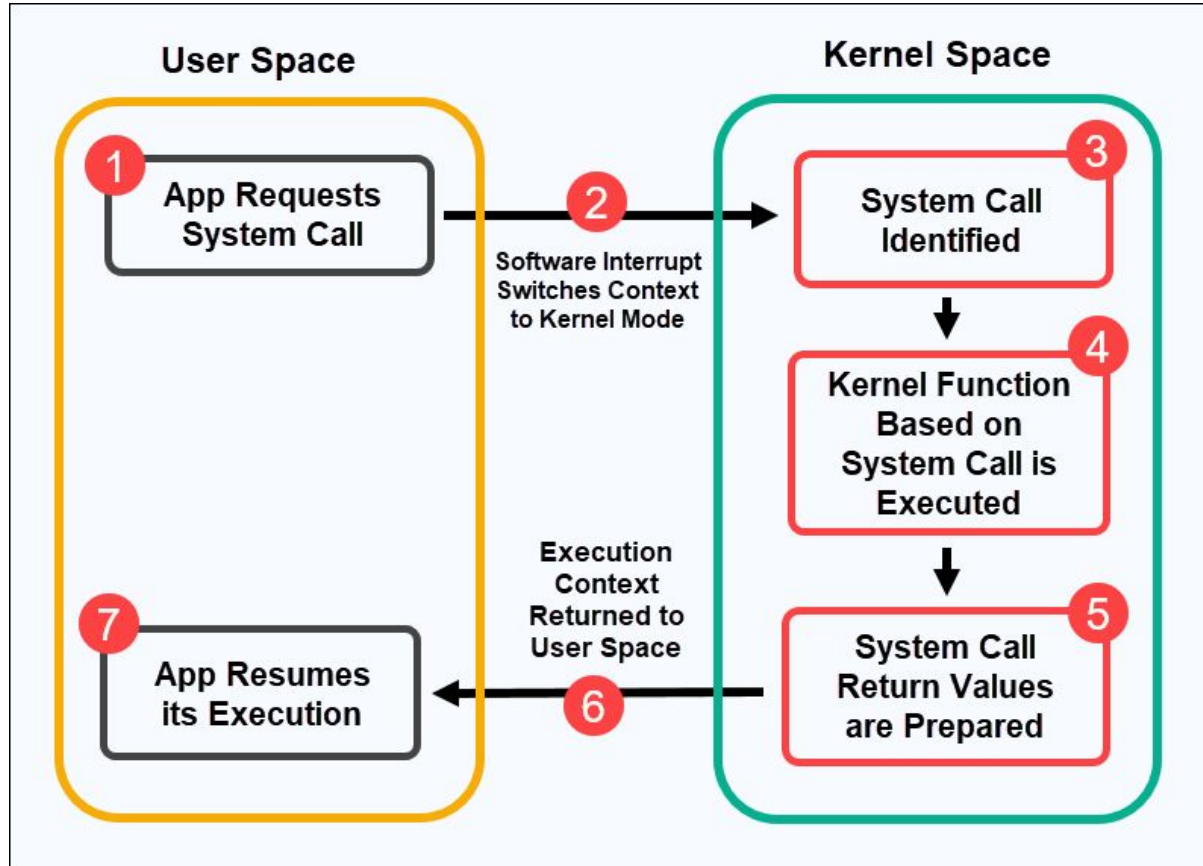
System calls

- System calls act as intermediaries between applications and the kernel. They provide an abstraction layer that shields essential system components, like the CPU and memory, from inadvertent errors and targeted attacks.
- A system call is an interface between a program running in user space and the operating system (OS).
- Application programs use system calls to request services and functionalities from the OS's kernel. This mechanism allows the program to call for a service, like reading from a file, without accessing system resources directly.

What Is the Purpose of the System Call?

- **User-Kernel Boundary.** System calls serve as the authorized gateway for user programs when requesting services from the kernel. They ensure that user programs cannot arbitrarily access kernel functions or critical system resources.
- **Resource Management.** User programs can request and manage vital resources like CPU time, memory, and file storage via system calls. The OS oversees the process and guarantees that it is completed in an organized manner.
- **Streamlined Development.** System calls abstract the complexities of hardware. This allows developers to perform operations like reading and writing to a file or managing network data without needing to write hardware-specific code.
- **Security and Access Control.** System calls implement checks to ensure that requests made by user programs are valid and that the programs have the necessary permissions to perform the requested operations.
- **Inter-Process Communication (IPC).** System calls provide the mechanisms for processes to communicate with each other. They offer features like pipes, message queues, and shared memory to facilitate this inter-process communication.
- **Network Operations.** System calls provide the framework for network communications between programs. Developers can devote their attention to building their application's logic instead of focusing on low-level network programming.

How Do System Calls Work?



How Do System Calls Work?

1. **System Call Request.** The application requests a system call by invoking its corresponding function. For instance, the program might use the `read()` function to read data from a file.
2. **Context Switch to Kernel Space.** A software interrupt or special instruction is used to trigger a context switch and transition from the user mode to the kernel mode.
3. **System Call Identified.** The system uses an index to identify the system call and address the corresponding kernel function.
4. **Kernel Function Executed.** The kernel function corresponding to the system call is executed. For example, reading data from a file.
5. **System Prepares Return Values.** After the kernel function completes its operation, any return values or results are prepared for the user application.
6. **Context Switch to User Space.** The execution context is switched back from kernel mode to user mode.
7. **Resume Application.** The application resumes its execution from where it left off, now with the results or effects of the system call.

What Are the Features of System Calls?

- **Security.** System calls ensure that user-space applications cannot harm the system or interfere with other processes.
- **Abstraction.** For example, programs do not need to know the specifics of network hardware configurations to send data over the internet or disk operations to read a file, as the OS handles these tasks.
- **Access Control.** System calls enforce security measures by checking whether a program has the appropriate permissions to access resources.
- **Consistency.** Interactions between the OS and program remain consistent, regardless of the underlying hardware configuration. The same program can run on different hardware if the operating system supports it.
- **Synchronous Operation.** Many system calls operate synchronously, blocking the calling process until the operation is complete. However, there are also asynchronous system calls that allow processes to continue execution without waiting.
- **Process Control.** System calls facilitate stable process management and multitasking through process creation, termination, scheduling, and synchronization mechanisms.
- **File Management.** System calls support file operations such as reading, writing, opening, and closing files.

What Are the Features of System Calls?

Device Management. System calls enable processes to request device access, perform read or write operations on these devices, and release them afterward.

Resource Management. System calls help allocate and deallocate resources like memory, CPU time, and I/O devices.

Maintenance. System calls are used to obtain or configure system information, such as the date and time or process status.

Communication. System calls allow processes to communicate with each other and synchronize their actions.

Error Handling. When a system call cannot be completed, it returns an error code that the calling program can process.

Types of System Calls

1. Process Control
2. File Management
3. Device Management
4. Information Maintenance
5. Communication
6. Security and Access Control

The table below lists common Unix and Windows system calls and their descriptions. Process control

UNIX SYSTEM CALLS	DESCRIPTION	WINDOWS API CALLS	DESCRIPTION
fork()	Create a new process.	CreateProcess()	Create a new process.
exit()	Terminate the current process.	ExitProcess()	Terminate the current process.
wait()	Make a process wait until its child processes terminate.	WaitForSingleObject()	Wait for a process or thread to terminate.
exec()	Execute a new program in a process.	CreateProcess() or ShellExecute()	Execute a new program in a new process.
getpid()	Get the unique process ID.	GetCurrentProcessId()	Get the unique process

Windows example

```
#include <windows.h>
#include <stdio.h>
int main() {
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));
    // Create a new process
    if (CreateProcess(NULL, "cmd.exe", NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
        printf("Notepad process created successfully!\n");
        CloseHandle(pi.hProcess);
        CloseHandle(pi.hThread);
    } else {
        printf("Error creating process: %d\n", GetLastError());
    }
    return 0;
}
```

Linux Example

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child process
        printf("Hello from child process!\n");
    } else if (pid > 0) {
        // Parent process
        printf("Hello from parent process!\n");
        wait(NULL); // Wait for the child process to finish
    } else {
        perror("fork() failed");
    }

    return 0;
}
```

File Management

<code>open()</code>	Open a file (or device).	<code>CreateFile()</code>	Open or create a file or device.
<code>close()</code>	Close an open file (or device).	<code>CloseHandle()</code>	Close an open object handle.
<code>read()</code>	Read from a file (or device).	<code>ReadFile()</code>	Read data from a file or input device.
<code>write()</code>	Write to a file (or device).	<code>WriteFile()</code>	Write data to a file or output device.
<code>lseek()</code>	Change the read/write location in a file.	<code>SetFilePointer()</code>	Set the position of the file pointer.
<code>unlink()</code>	Delete a file.	<code>DeleteFile()</code>	Delete an existing file.

Directory Management

mkdir()	Create a new directory.	CreateDirectory()	Create a new directory.
rmdir()	Remove a directory.	RemoveDirectory()	Remove an existing directory.
chdir()	Change the current directory.	SetCurrentDirectory())	Change the current directory.
stat()	Get file status.	GetFileAttributesEx())	Get extended file attributes.
fstat()	Get status of an open file.	GetFileInformationByHandle()	Get file information using a file handle.
link()	Create a link to a file.	CreateHardLink()	Create a hard link to an existing file.
symlink()	Get the status of an open file.	CreateSymbolicLink())	Create a symbolic link.

brk() or sbrk()	Increase/decrease the program's data space.	VirtualAlloc() or VirtualFree()	Reserve, commit, or free a region of memory.
mmap()	Map files or devices into memory.	MapViewOfFile()	Map a file into the application's address space.
Information Maintenance			
time()	Get the current time.	GetSystemTime()	Get the current system time.
alarm()	Get the status of an open file.	SetWaitableTimer()	Set a timer object.
getuid()	Set an alarm clock for the delivery of a signal.	GetUserName() or LookupAccountName()	Get the username or ID.
getgid()	Get the group ID.	GetTokenInformation()	Get the group information of a security token.

Communication Calls

socket()	Create a new socket.	socket()	Create a new socket.
bind()	Bind a socket to a network address.	bind()	Bind a socket to a network address.
listen()	Bind a socket to a network address.	listen()	Listen for connections on a socket.
accept()	Accept a new connection on a socket.	accept()	Accept a new connection on a socket.
connect()	Initiate a connection on a socket.	connect()	Initiate a connection on a socket.
send() or recv()	Send and receive data on a socket.	send() or recv()	Send and receive data on a socket.

Security and Access Control

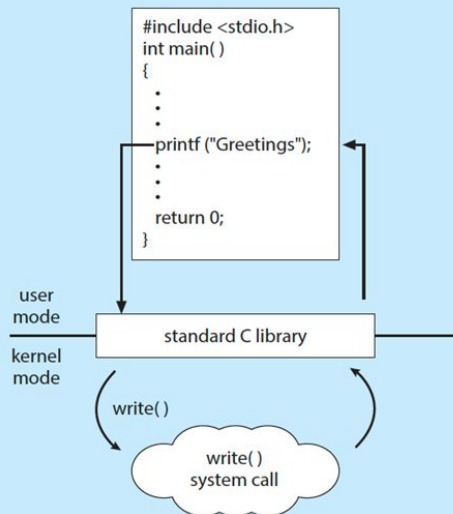
chmod() or umask()	Change the permissions/mode of a file.	SetFileAttributes() or SetSecurityInfo()	Change the file attributes or security info.
chown()	Change the owner and group of a file.	SetSecurityInfo()	Set the security information.

Standard C Library Example

C program invoking `printf()` library call, which calls `write()` system call

THE STANDARD C LIBRARY

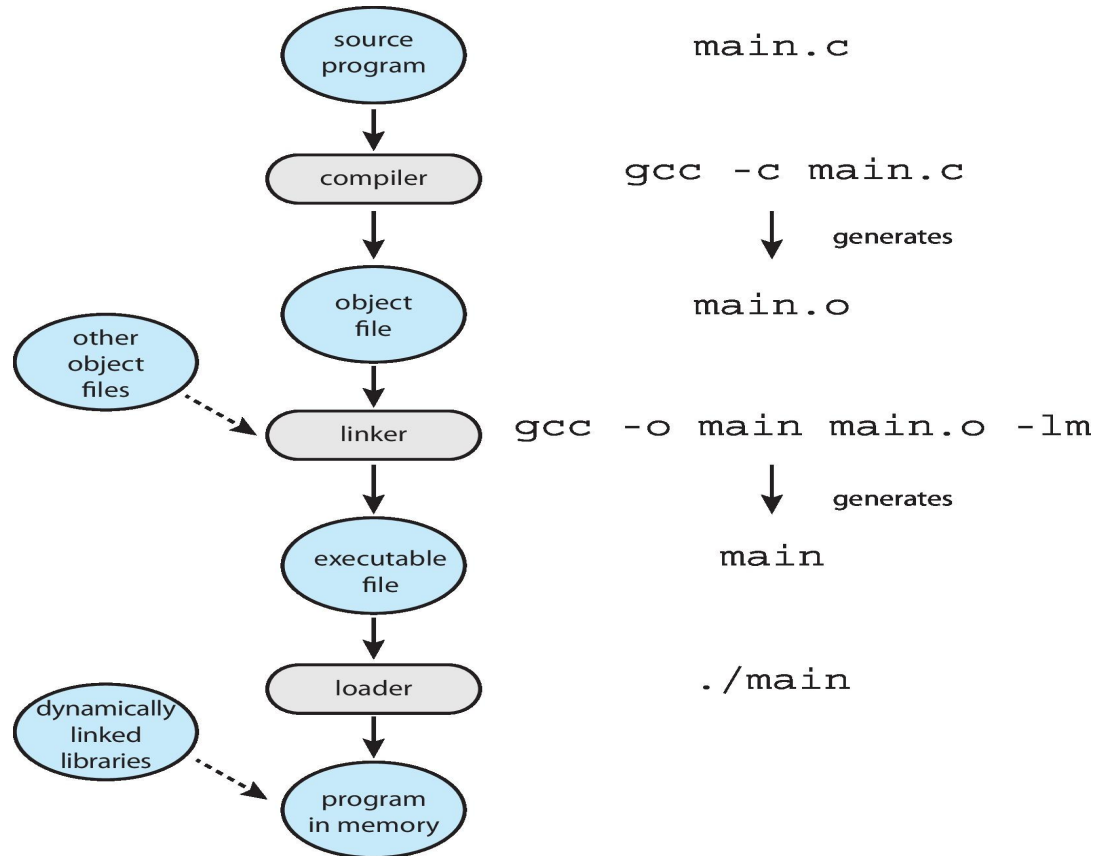
The standard C library provides a portion of the system-call interface for many versions of UNIX and Linux. As an example, let's assume a C program invokes the `printf()` statement. The C library intercepts this call and invokes the necessary system call (or calls) in the operating system—in this instance, the `write()` system call. The C library takes the value returned by `write()` and passes it back to the user program:



Linkers and Loaders

- Source code compiled into object files designed to be loaded into any physical memory location
relocatable object file
- Linker combines these into single binary executable file
 - Also brings in libraries
- Program resides on secondary storage as binary executable
- Must be brought into memory by loader to be executed
 - Relocation assigns final addresses to program parts and adjusts code and data in program to match those addresses
- Modern general purpose systems don't link libraries into executables
 - Rather, dynamically linked libraries (in Windows, DLLs) are loaded as needed, shared by all that use the same version of that same library (loaded once)
- Object, executable files have standard formats, so operating system knows how to load and start them

The Role of the Linker and Loader



Types of Kernel

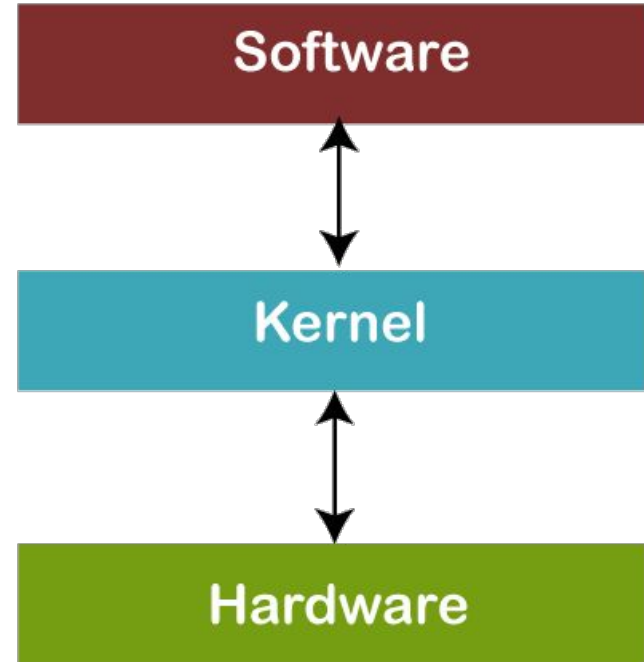
1. Monolithic Kernel
2. Micro Kernel
3. Hybrid Kernel
4. Nanokernel
5. Exokernel

Monolithic Kernel

In a monolithic kernel, the same memory space is used to implement user services and kernel services.

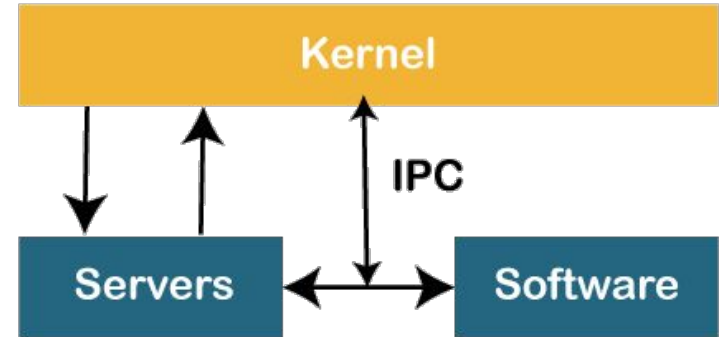
Examples:

Unix, Linux, Open VMS, XTS-400, etc.



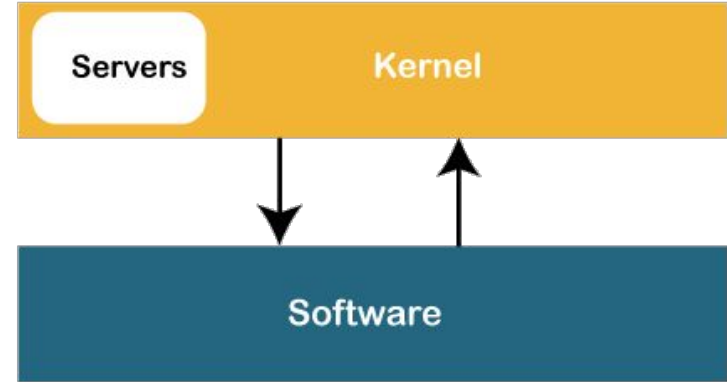
Microkernel

- A microkernel is also referred to as μK , and it is different from a traditional kernel or Monolithic Kernel.
- In this, user services and kernel services are implemented into two different address spaces: user space and kernel space.
- Since it uses different spaces for both the services, so, the size of the microkernel is decreased, and which also reduces the size of the OS.
- Example: L4, AmigaOS, Minix, K42, etc.



Hybrid Kernel

- Hybrid kernels are also known as modular kernels, and it is the combination of both Monolithic and Microkernels.
- It takes advantage of the speed of monolithic kernels and the modularity of microkernels.
- A hybrid kernel can be understood as the extended version of a microkernel with additional properties of a monolithic kernel.
- These kernels are widely used in commercial OS, such as different versions of MS Windows.



Nanokernel

- The complete code of the kernel is very small, which means the code executing in the privileged mode of the hardware is very small.
- Here the term ***nano*** defines a kernel that supports a nanosecond clock resolution.
- Examples of Nanokernel are EROS etc.

Exokernel

- Exokernel is still developing and is the experimental approach for designing OS.
- This type of kernel is different from other kernels as in this resource protection is kept separated from management, which allows us to perform application - specific customization.

Implementation

- Much variation
 - Early OSes in assembly language
 - Then system programming languages like Algol, PL/1
 - Now C, C++
- Actually usually a mix of languages
 - Lowest levels in assembly
 - Main body in C
 - Systems programs in C, C++, scripting languages like PERL, Python, shell scripts
- More high-level language easier to port to other hardware
 - But slower
- Emulation can allow an OS to run on non-native hardware

Operating system structure

General-purpose OS is very large program

Various ways to structure ones

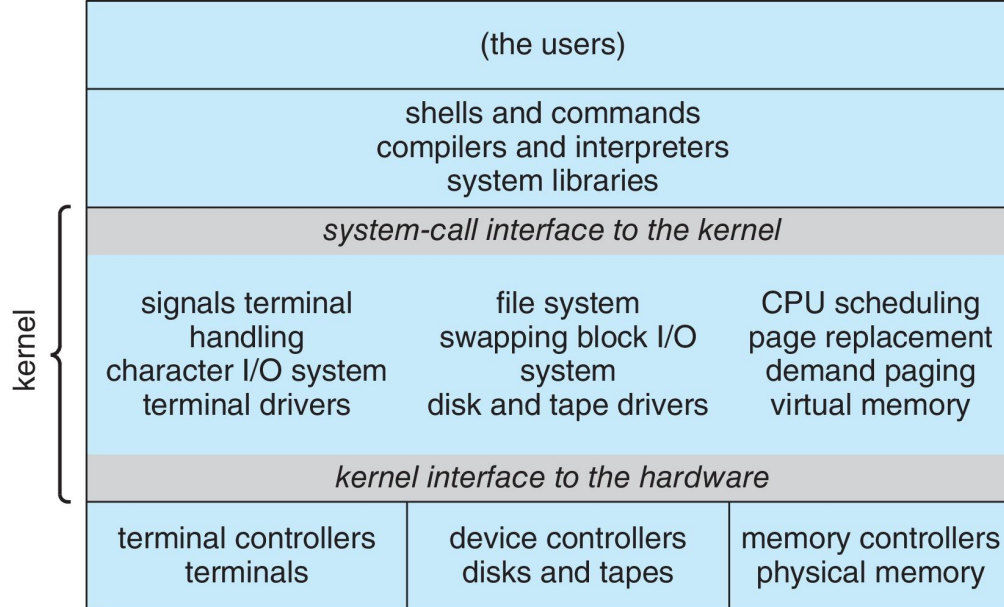
- Simple structure – MS-DOS
- More complex – UNIX
- Layered – an abstraction
- Microkernel – Mach

Monolithic Structure – Original UNIX

- UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring.
- The UNIX OS consists of two separable parts
 - Systems programs
 - The kernel
 - Consists of everything below the system-call interface and above the physical hardware
 - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level

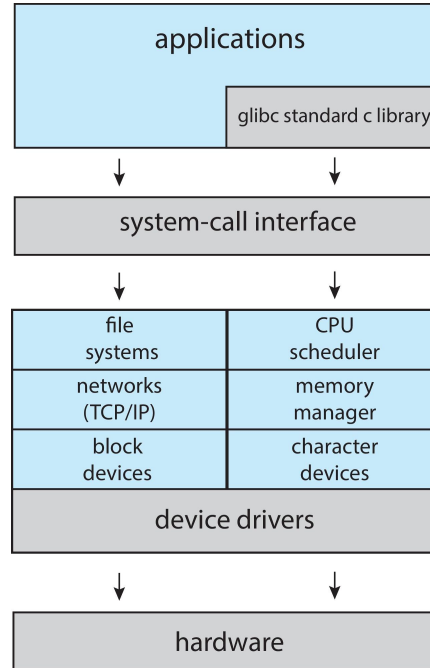
Traditional UNIX System Structure

Beyond simple but not fully layered



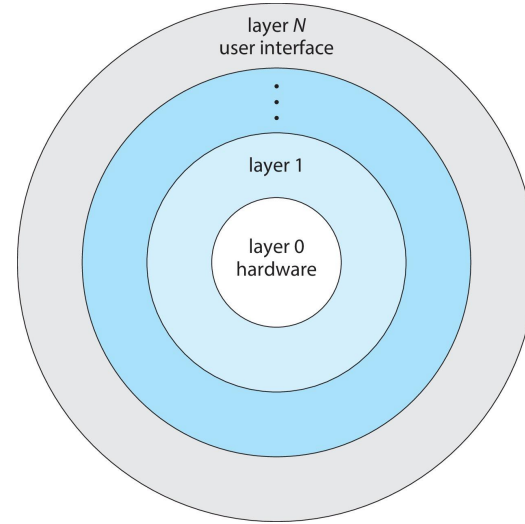
Linux System Structure

Monolithic plus modular design



Layered Approach

- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers

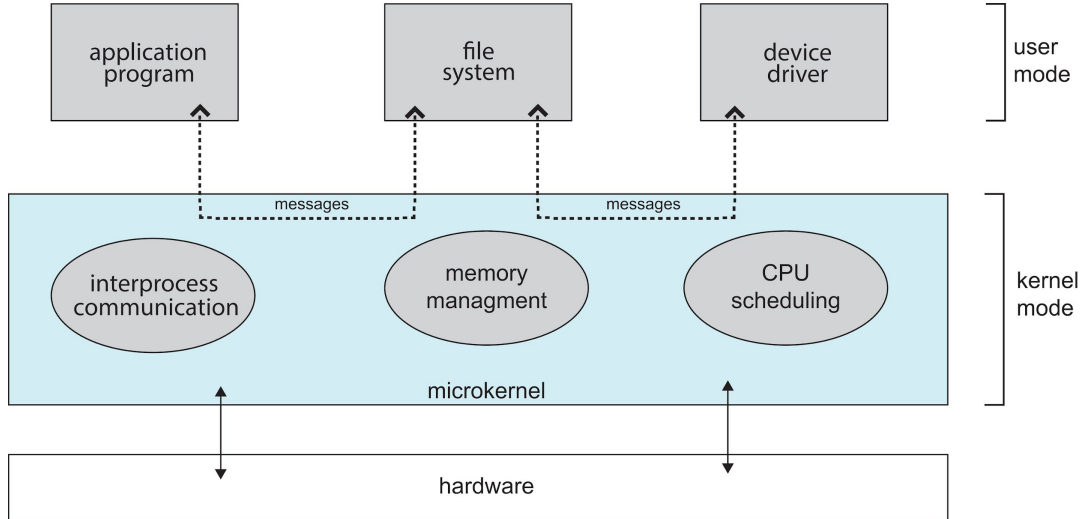


Microkernels

Mach is a kernel developed at Carnegie Mellon University by Richard Rashid and Avie Tevanian to support **operating system** research

- Moves as much from the kernel into user space
- **Mach** is an example of **microkernel**
 - Mac OS X kernel (**Darwin**) partly based on Mach
- Communication takes place between user modules using **message passing**
- Benefits:
 - Easier to extend a microkernel
 - Easier to port the operating system to new architectures
 - More reliable (less code is running in kernel mode)
 - More secure
- Detriments:
 - Performance overhead of user space to kernel space communication

Microkernel System Structure



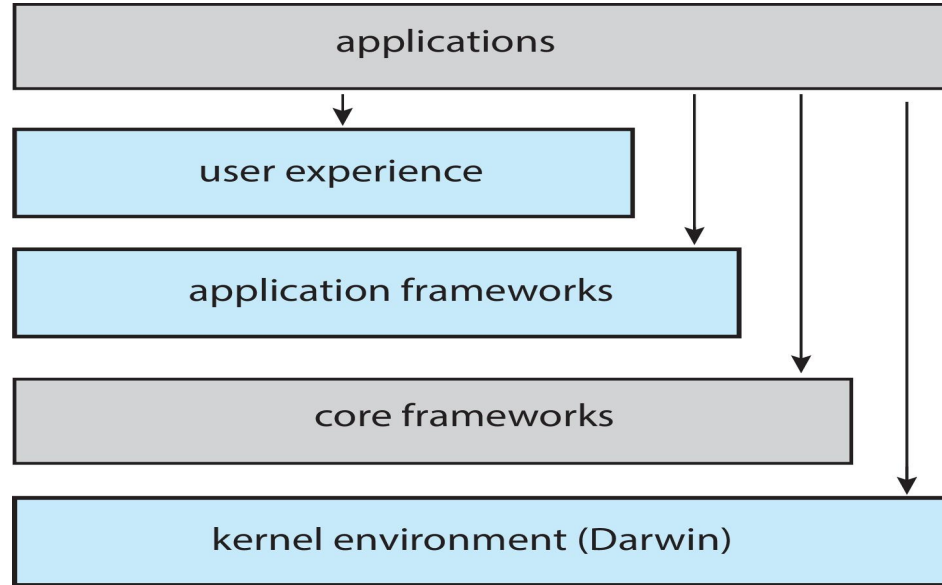
Modules

- Many modern operating systems implement **loadable kernel modules (LKMs)**
 - Uses object-oriented approach
 - Each core component is separate
 - Each talks to the others over known interfaces
 - Each is loadable as needed within the kernel
- Overall, similar to layers but with more flexible
 - Linux, Solaris, etc.

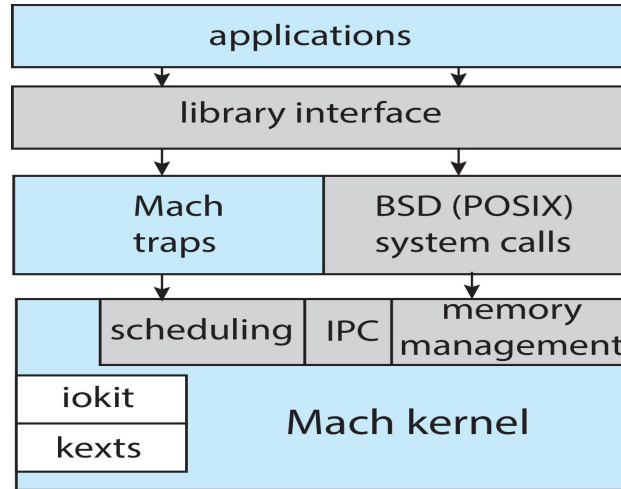
Hybrid Systems

- Most modern operating systems are not one pure model
 - Hybrid combines multiple approaches to address performance, security, usability needs
 - Linux and Solaris kernels in kernel address space, so monolithic, plus modular for dynamic loading of functionality
 - Windows mostly monolithic, plus microkernel for different subsystem *personalities*
- Apple Mac OS X hybrid, layered, **Aqua** UI plus **Cocoa** programming environment
 - Below is kernel consisting of Mach microkernel and BSD Unix parts, plus I/O kit and dynamically loadable modules (called **kernel extensions**)

macOS and iOS Structure



Darwin



Virtual machines

